```
      default-filters:
        - SaveSession      ❶
```

❶ Ensures the session data is saved before forwarding a request downstream

That's a critical point when Spring Session is combined with Spring Security. Chapters 11 and 12 will cover more details about session management. For now, let's set up an integration test to verify the Spring context in Edge Service loads correctly, including the integration with Redis.

The approach we'll use is similar to the one we used to define PostgreSQL test containers in the previous chapter. Let's extend the existing `EdgeServiceApplicationTests` class generated by Spring Initializr and configure a Redis test container. For this example, it's enough to verify that the Spring context loads correctly when Redis is used for storing web session-related data.

Listing 9.19 Using a Redis container to test the Spring context loading

```
package com.polarbookshop.edgeservice;

import org.junit.jupiter.api.Test;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;

@SpringBootTest(                                                    ❶
  webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT
)
@Testcontainers                                                     ❷
class EdgeServiceApplicationTests {

   private static final int REDIS_PORT = 6379;

   @Container
   static GenericContainer<?> redis =                               ❸
      new GenericContainer<>(DockerImageName.parse("redis:7.0"))
       .withExposedPorts(REDIS_PORT);
```